EECS 836: Machine Learning

Zijun Yao

Assistant Professor, EECS Department

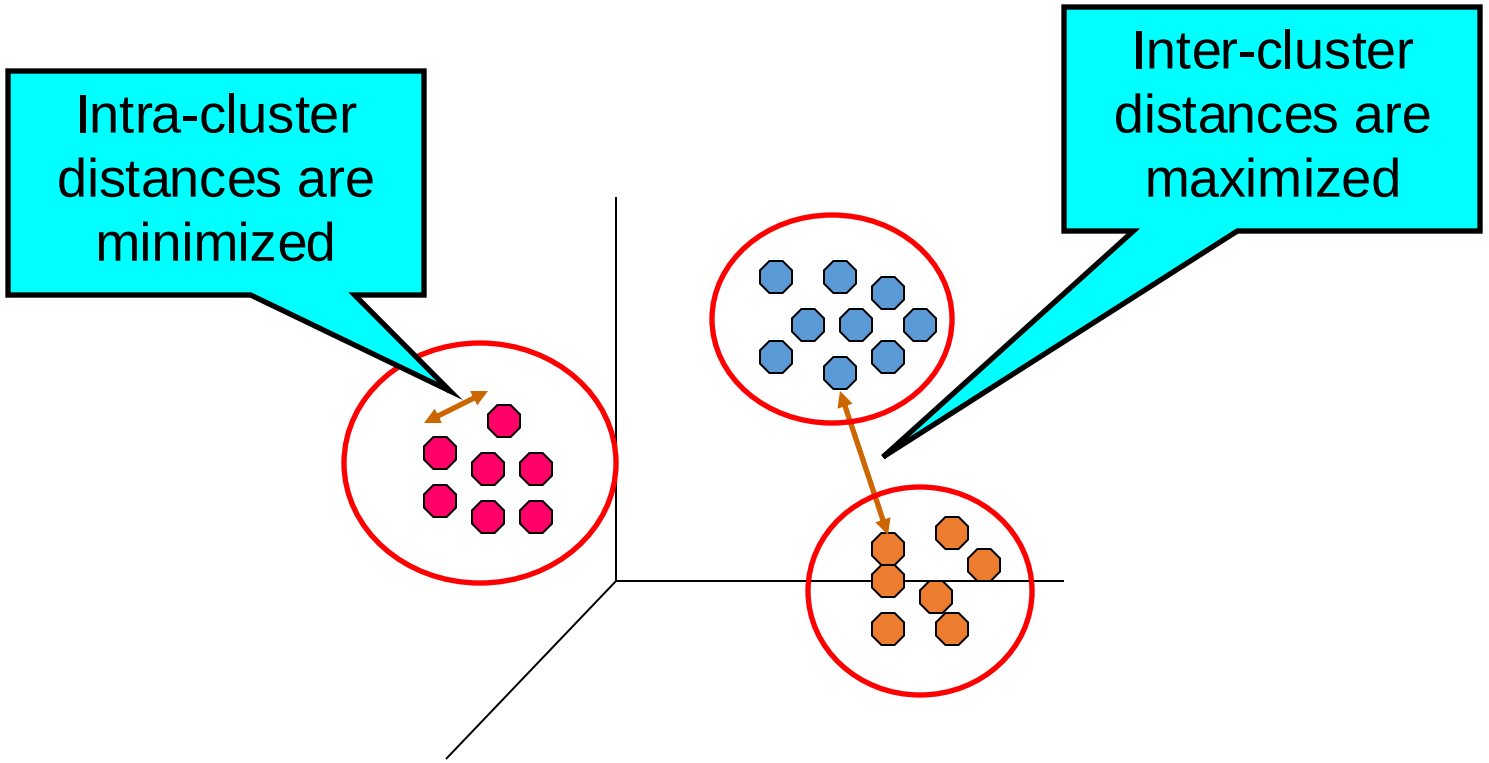
The University of Kansas

Agenda

- K-means Clustering
  - K-means Algorithm
  - K-means Limitations
  - EM Algorithm and Relation to k-means

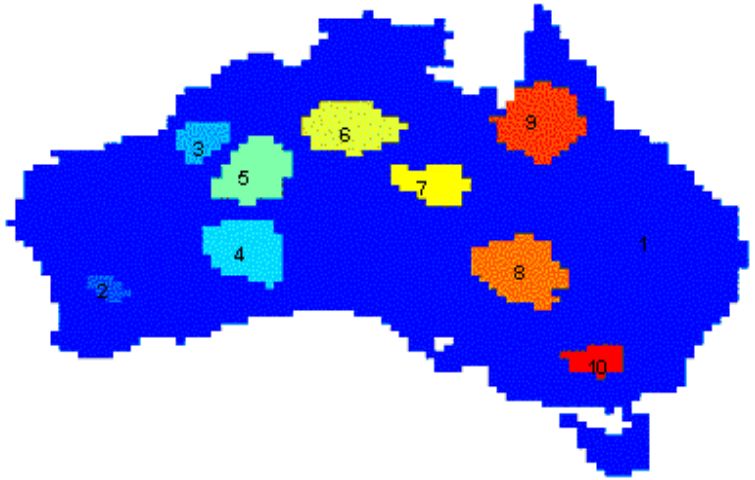
What is clustering?

- Given a set of objects, grouping them such that
  - The objects in the same group (cluster) are similar (or related) to one another
  - The objects are different from (or unrelated to) the objects in other groups



Unsupervised learning

- A common form of unsupervised learning
  - Learning from raw (unlabeled, unannotated, etc) data, as opposed to supervised data where a classification of examples is given
- An important task that finds many applications in science, engineering, information technology, and other places



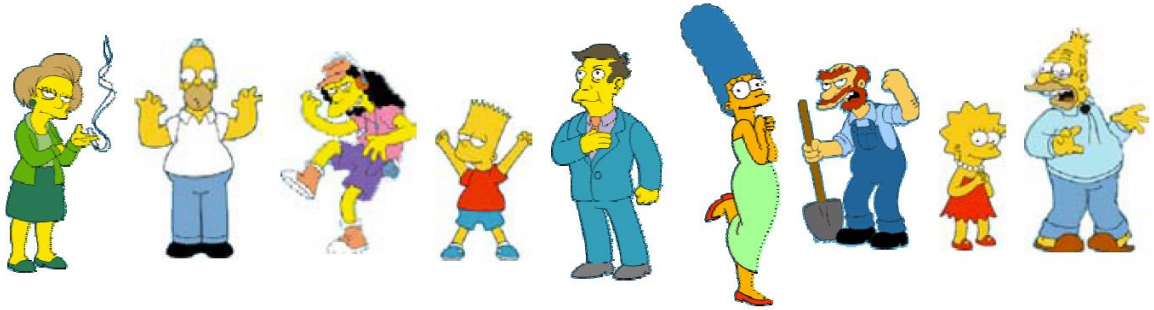
Clustering precipitation in Australia

	Discovered Clusters	*Industry Group*
1	Applied-Matl-DOWN,Bay-Network-Down,3-COM-DOWN, Cabletron-Sys-DOWN,CISCO-DOWN,HP-DOWN, DSC-Comm-DOWN,INTEL-DOWN,LSI-Logic-DOWN, Micron-Tech-DOWN,Texas-Inst-Down,Tellabs-Inc-Down, Natl-Semiconduct-DOWN,Oracl-DOWN,SGI-DOWN, Sun-DOWN	Technology1-DOWN
2	Apple-Comp-DOWN,Autodesk-DOWN,DEC-DOWN, ADV-Micro-Device-DOWN,Andrew-Corp-DOWN, Computer-Assoc-DOWN,Circuit-City-DOWN, Compaq-DOWN, EMC-Corp-DOWN, Gen-Inst-DOWN, Motorola-DOWN,Microsoft-DOWN,Scientific-Atl-DOWN	Technology2-DOWN
3	Fannie-Mae-DOWN,Fed-Home-Loan-DOWN, MBNA-Corp-DOWN,Morgan-Stanley-DOWN	Financial-DOWN
4	Baker-Hughes-UP,Dresser-Inds-UP,Halliburton-HLD-UP, Louisiana-Land-UP,Phillips-Petro-UP,Unocal-UP, Schlumberger-UP	Oil-UP

Clustering stocks by industry and return

Cluster exists in many data types

- People



- Images

- Language

- Species

Notion of a cluster can be ambiguous

How many clusters?

Six Clusters

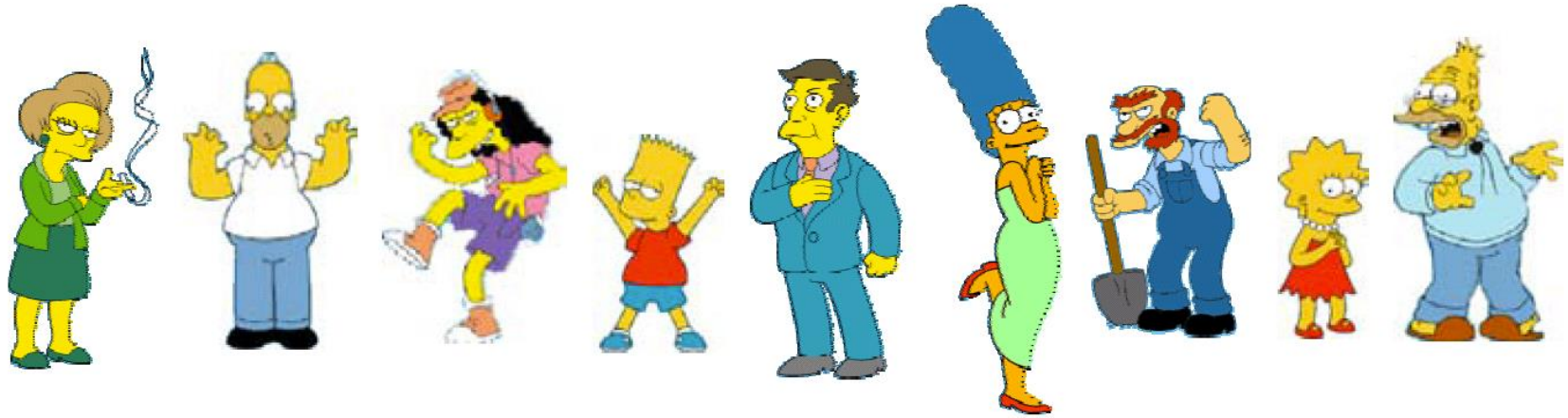
Two Clusters

Four Clusters

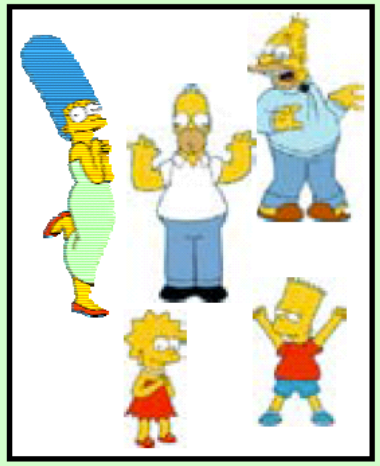
Issues for clustering

- What is a natural grouping among these objects?
  - Definition of "groupness“
- What makes objects “related”?
  - Definition of "similarity/distance“
- How many clusters?
  - Fixed a priori?
  - Completely data driven?
    - Avoid “trivial” clusters - too large or small
- Clustering Algorithms
  - Partitional algorithms
  - Hierarchical algorithms

Define grouping among data objects

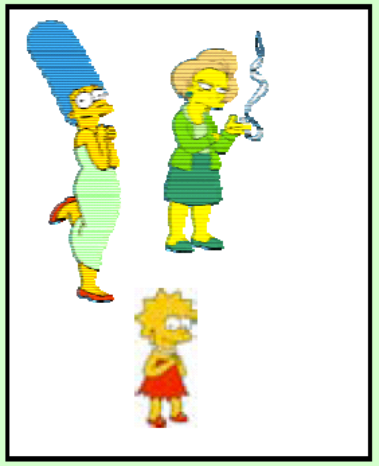


Clustering is subjective



Simpson's Family

School Employees

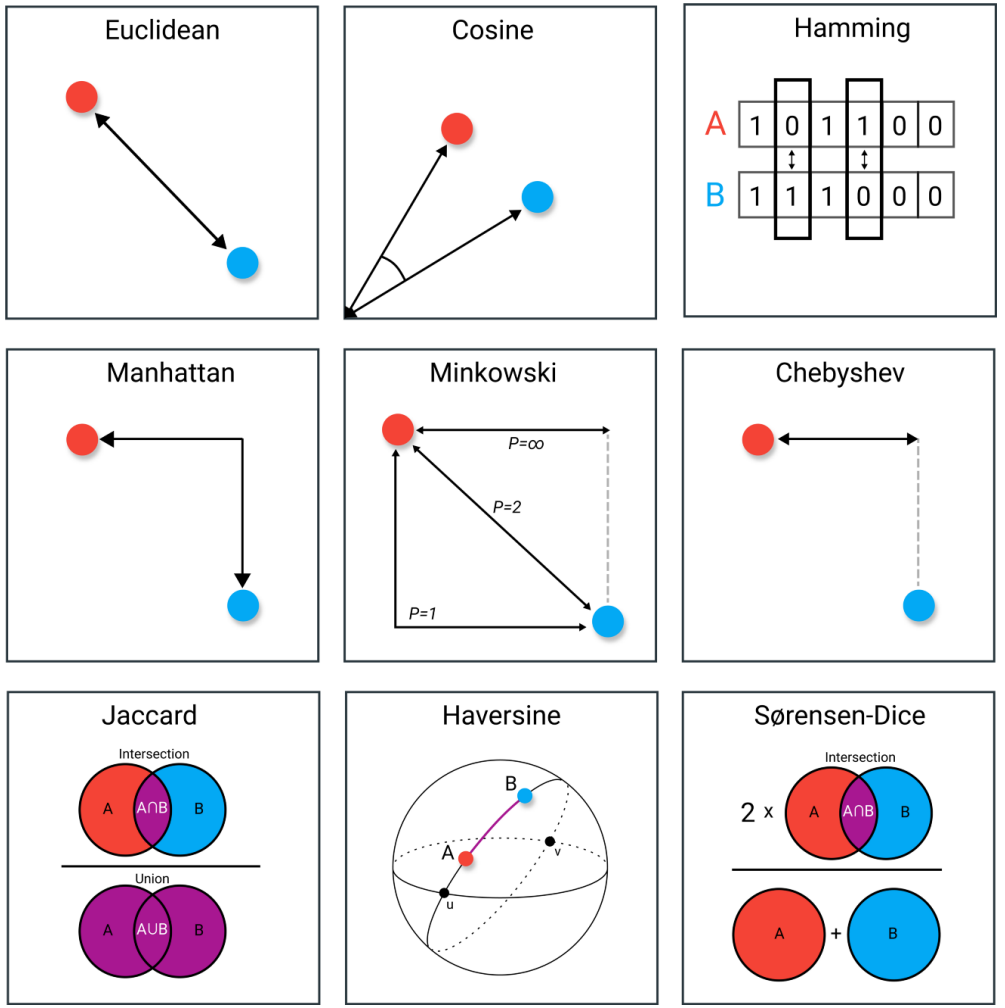


Females

Males

Define similarity among data objects

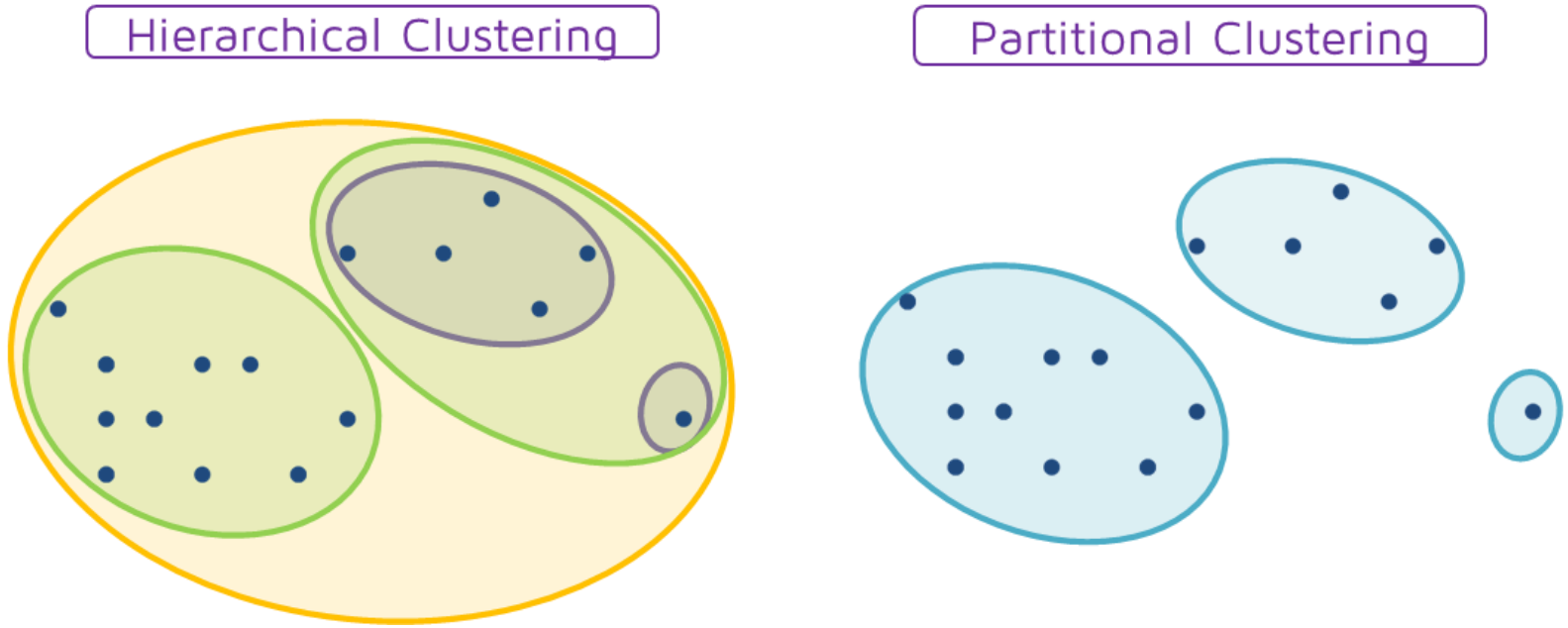
- Depends on representation of data, similarity measures the distance/proximity between vectors
- Choosing a similarity depends on particular tasks and data



https://towardsdatascience.com/9-distance-measures-in-data-science-918109d069fa

Types of clustering

- Partitional clustering
  - A division of data objects into non-overlapping subsets (clusters)
- Hierarchical clustering
  - A set of nested clusters organized as a hierarchical tree



Distinction between clustering

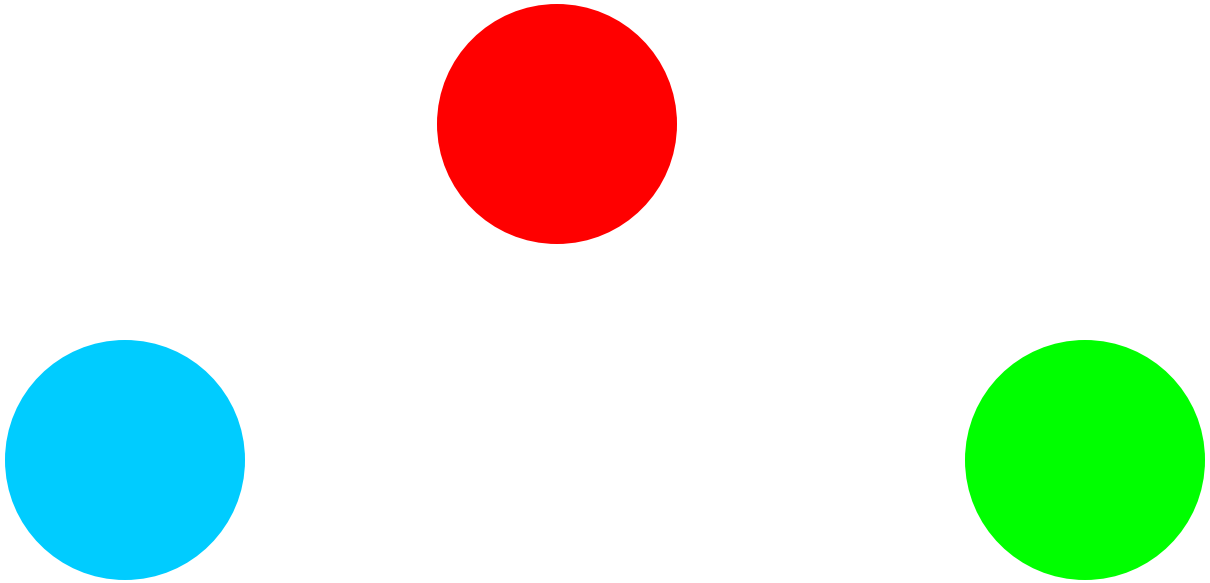
- Exclusive (or non-overlapping) vs. non-exclusive (overlapping)
  - In non-exclusive clustering, points may belong to multiple clusters.

- Fuzzy (or soft) clustering vs. non-fuzzy (or hard)
  - In fuzzy clustering, a point belongs to every cluster with some weight (or probability) between 0 and 1
    - Weights must sum to 1

- Partial vs. complete
  - In some cases, we only want to cluster some of the data

Types of clusters: well-separated

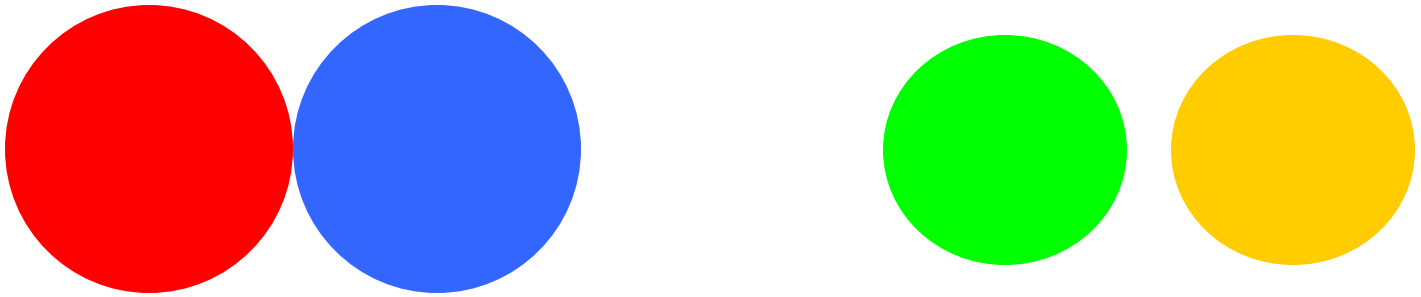
- Well-separated clusters:
  - A cluster is a set of points such that any point in a cluster is closer (or more similar) to every other point in the cluster than to any point not in the cluster.



3 well-separated clusters

Types of clusters: prototype-based

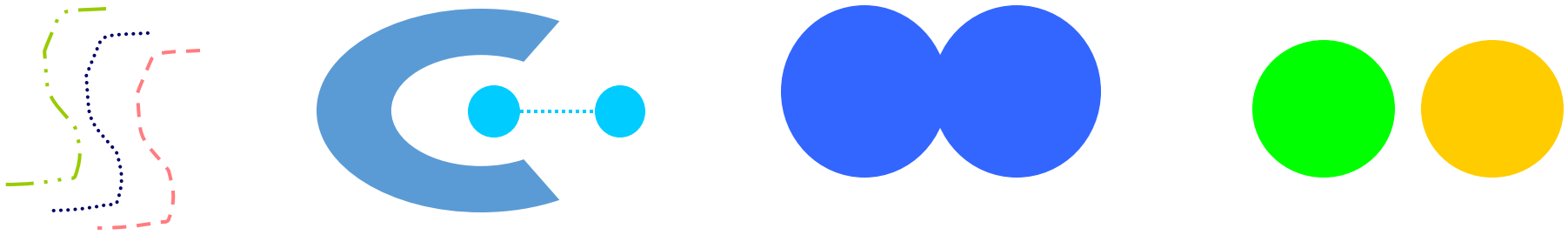
- Prototype-based (or center-based) clusters
  - A cluster is a set of objects such that object of a cluster is closer (more similar) to the prototype (center) of a cluster, than to the center of other clusters
  - Centroid, the average of all the points in the cluster
  - Medoid, the most “representative” point of a cluster



4 center-based clusters

Types of clusters: contiguity-based

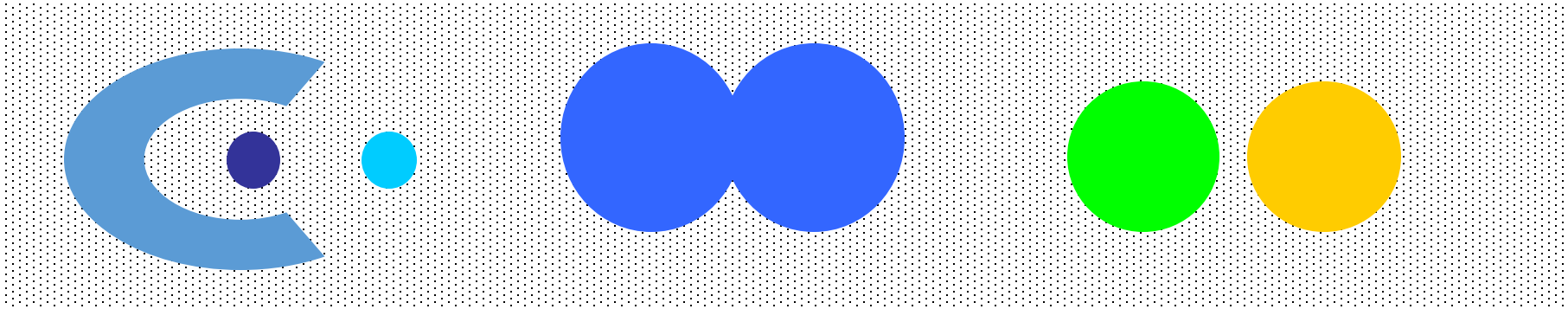
- Contiguous cluster (nearest neighbor or transitive)
  - A cluster is a set of points such that a point in a cluster is closer (or more similar) to at least one points in the cluster than to any point not in the cluster.



8 contiguous clusters

Types of clusters: density-based

- Density-based clusters
  - A cluster is a dense region of points, which is separated by low-density regions, from other regions of high density.
  - Used when the clusters are irregular or intertwined, and when noise and outliers are present.



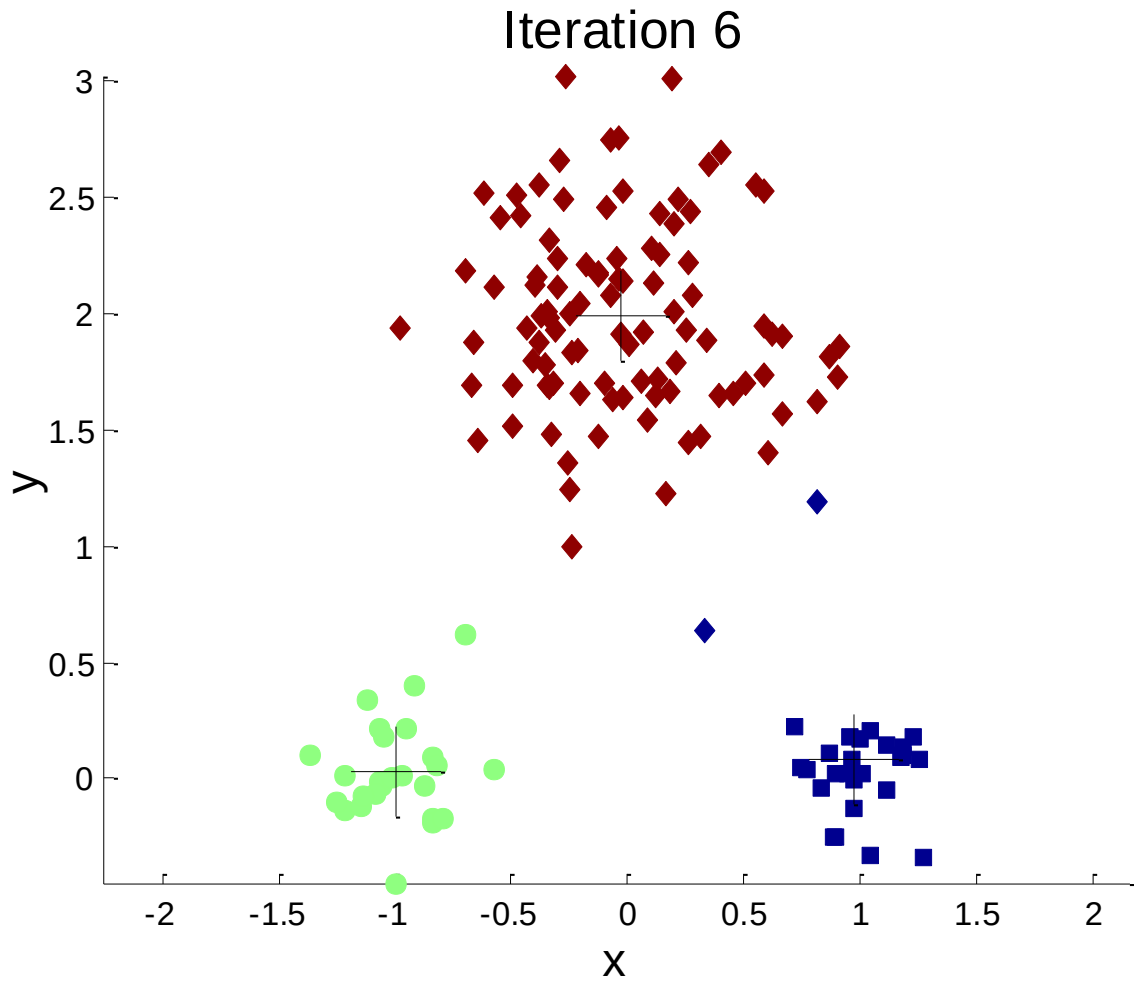
6 density-based clusters

K-means algorithm

- Partitional clustering approach
- Each cluster is associated with a centroid (center point)
- Each point is assigned to the cluster with the closest centroid
- Number of clusters, K, must be specified
- Objective
  - minimize the sum of distances of all the points to their respective centroid

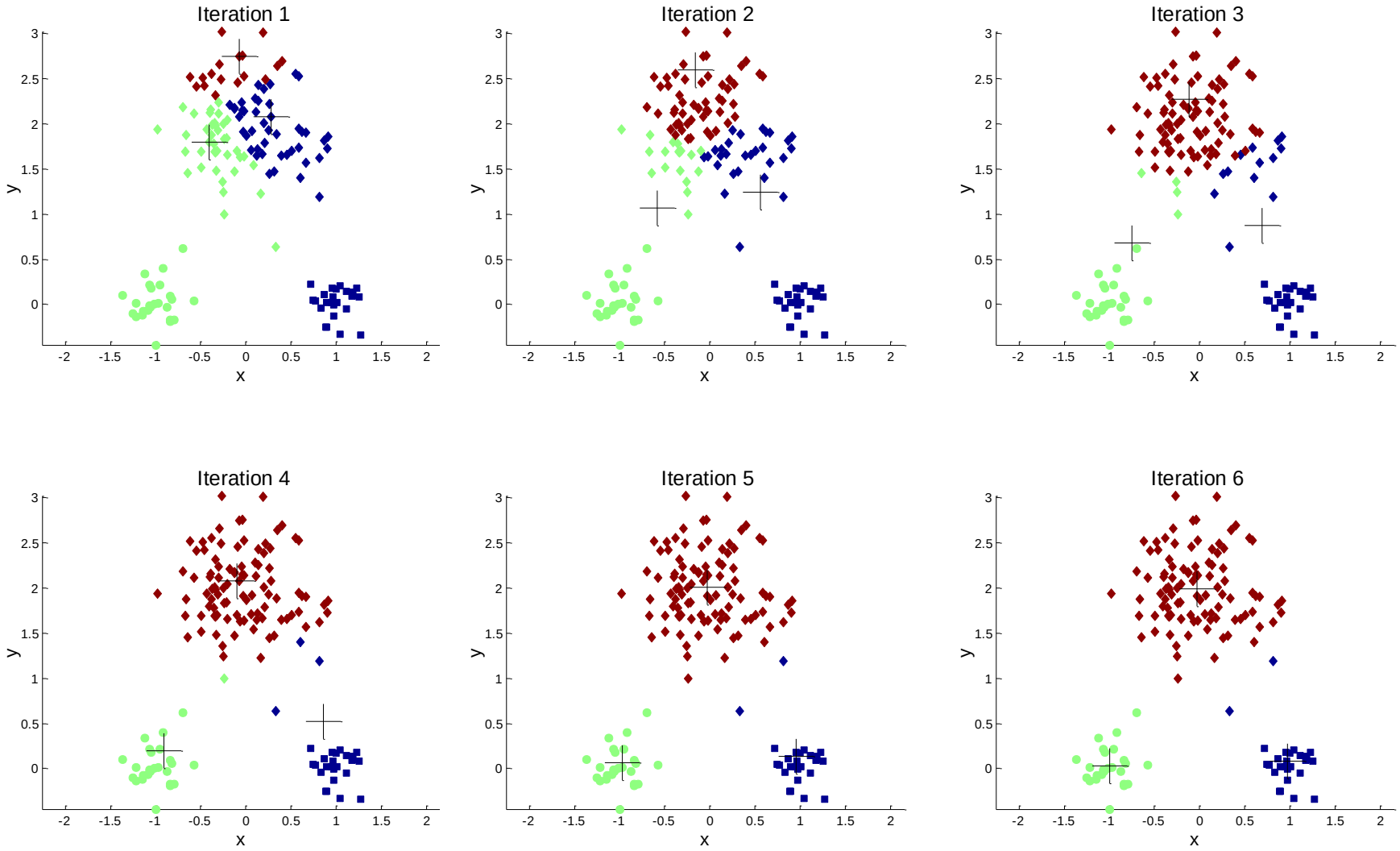
1. Select K points as the initial centroids
2. **repeat**
3. Form K clusters by assigning each point to the closest centroid
4. Compute the new centroid* of each cluster (distance minimizing)
5. **until** The centroids do not change

Example of k-means clustering



1. Select K points as the initial centroids
2. **repeat**
3. Form K clusters by assigning each point to the closest centroid
4. Compute the new centroid* of each cluster (distance minimizing)
5. **until** The centroids do not change

Example of k-means clustering



K-means algorithm details

- Simple iterative algorithm.
  - Repeat {assign each point to a nearest centroid; re-compute cluster centroids} until centroids stop changing.
- Initial centroids are often chosen randomly.
  - Clusters produced can vary from one run to another
- The centroid is (typically) the mean of the points in the cluster
  - Mean for SSE and cosine similarity
- Most of the convergence happens in the first few iterations.
  - Often the stopping condition is changed to "until relatively few points change clusters"
- Complexity is O(n * K * I * d)
  - n = number of points
  - K = number of clusters
  - I = number of iterations
  - d = number of attributes

K-means objective function

- A common objective function (used with Euclidean distance measure) is Sum of Squared Error (SSE)
  - For each point, the error is the distance to the nearest cluster center (centroid)
  - To get SSE, we square these errors and sum them.

$$SSE = \sum_{i=1}^{K} \sum_{x \in C_i} (x - c_i)^2$$

  - K is number of clusters
  - x is a data point in cluster C_i
  - c_i is the centroid (mean) for cluster C_i
  - SSE decreased in each iteration of K-means until it reaches a minima.

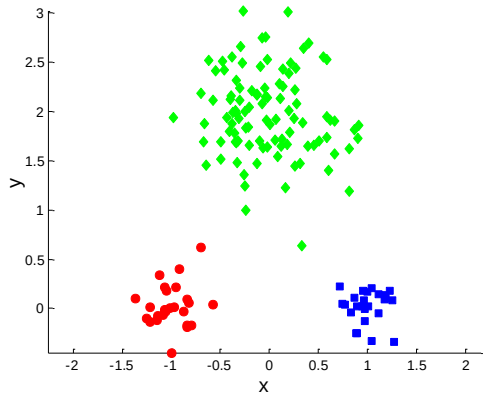
Problems with k-means

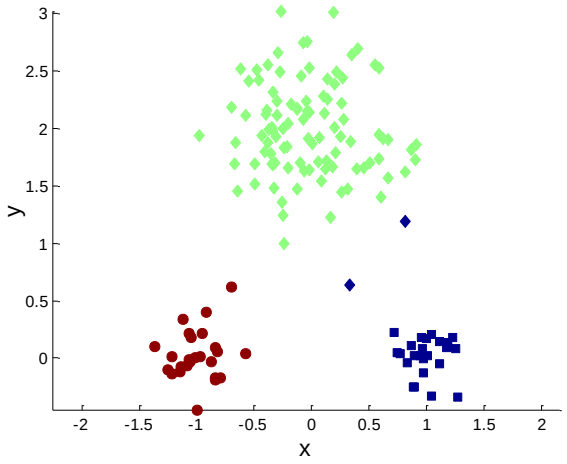
- Very sensitive to the initial points
  - Different initial centroids lead to various results
  - Seed the centroids using a better method than randomly choosing the centroids
- Must manually choose k
  - Learn the optimal k for the clustering
  - Note that this requires a performance measure

K-means algorithm – initialization

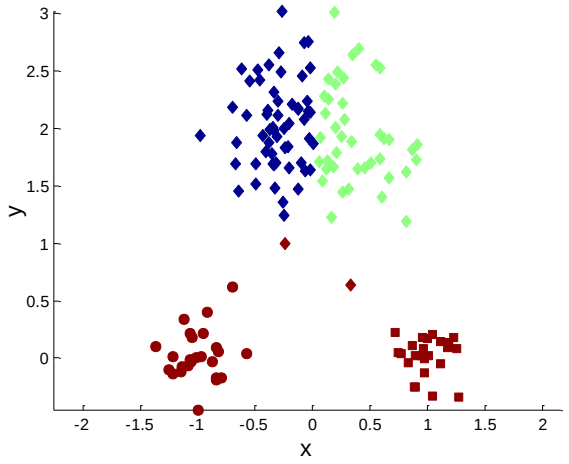
- Initial centroids are often chosen randomly
  - Clusters produced vary from one run to another.

Original Points



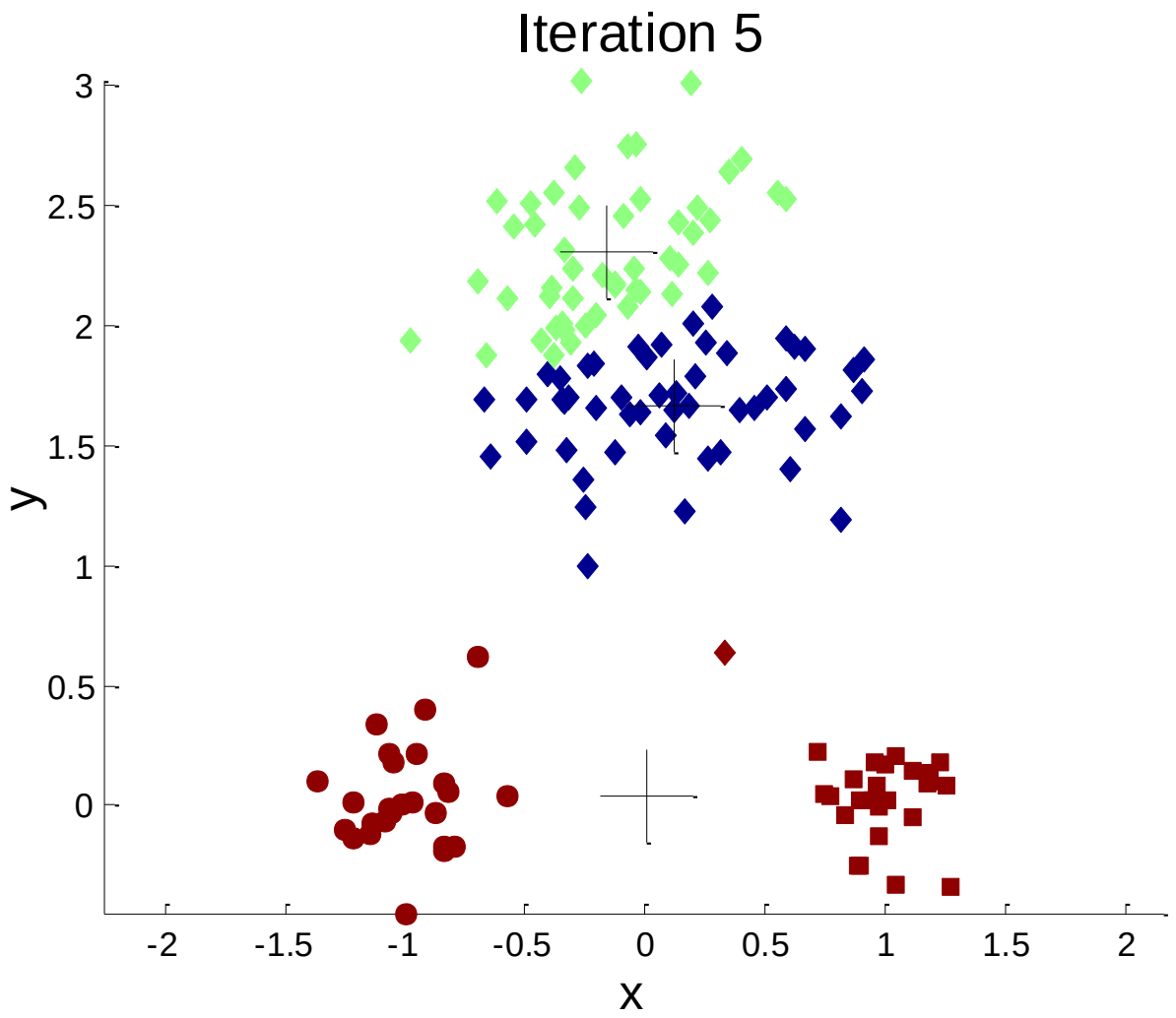


Optimal Clustering

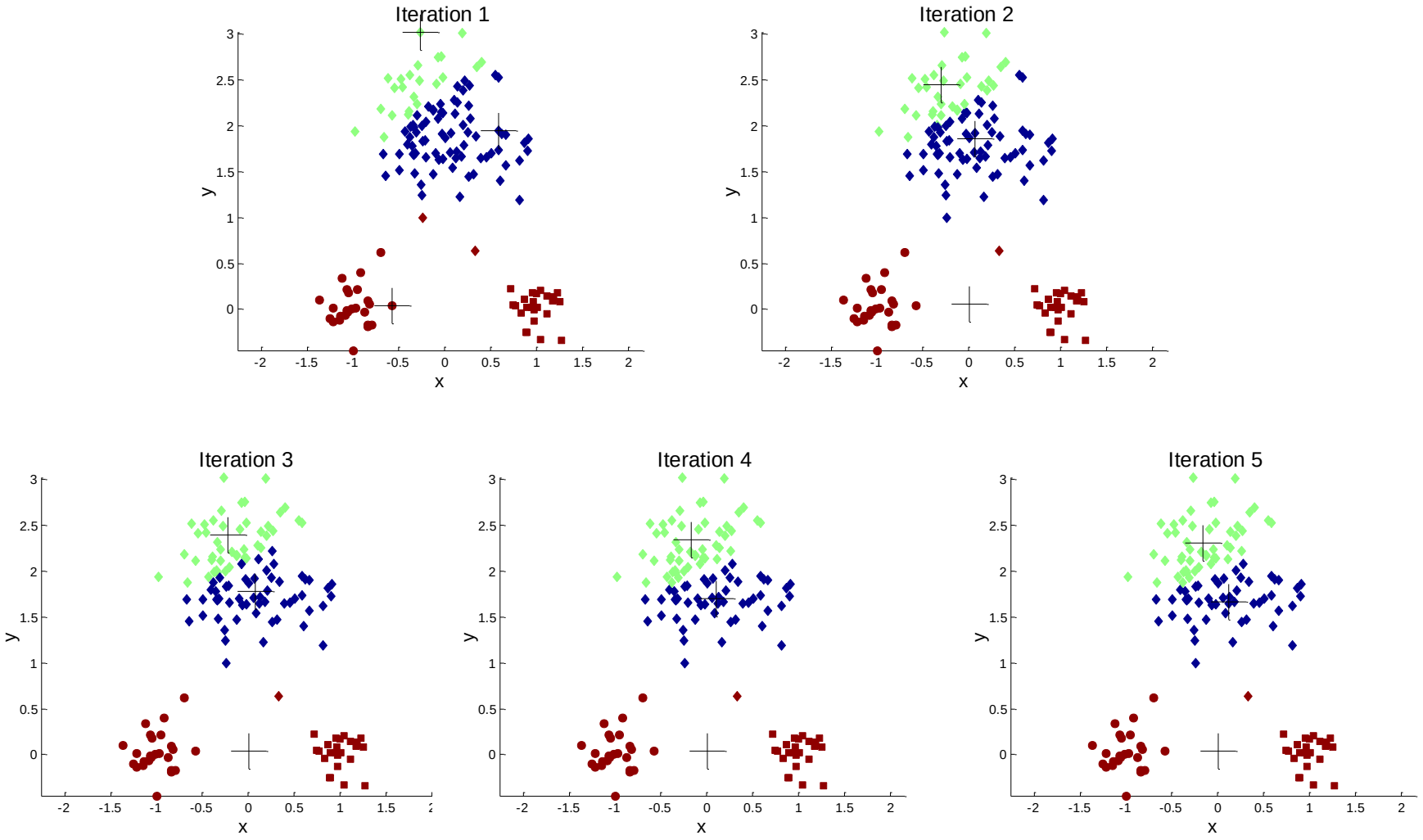


Sub-optimal Clustering

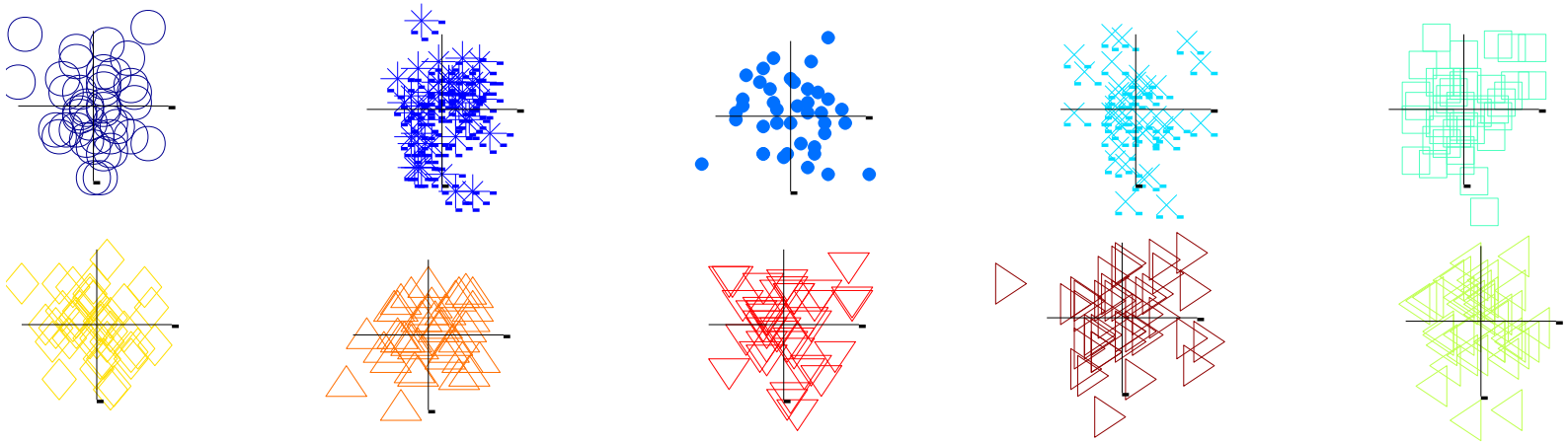
Importance of choosing initial centroids



Importance of choosing initial centroids

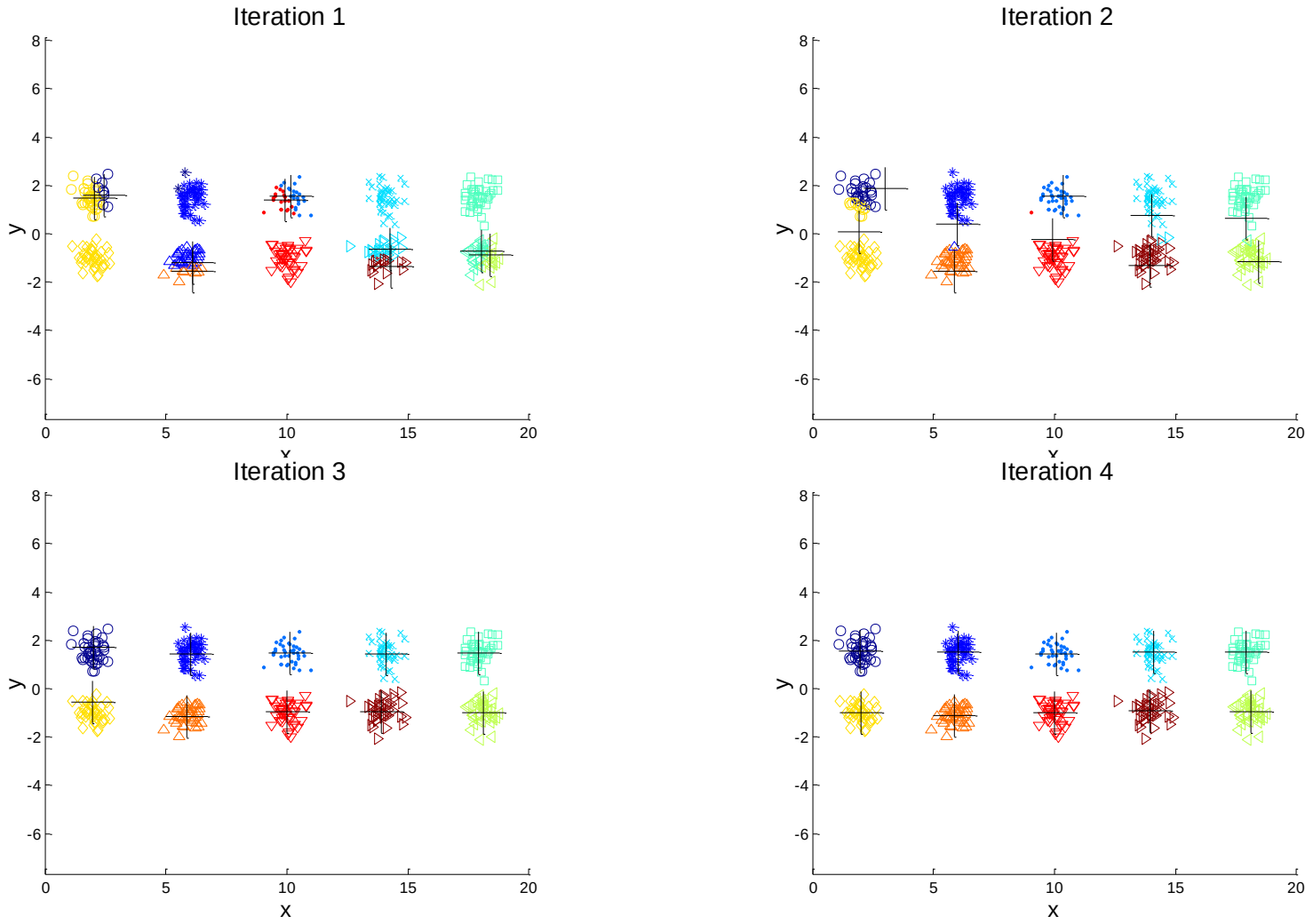


10 clusters example



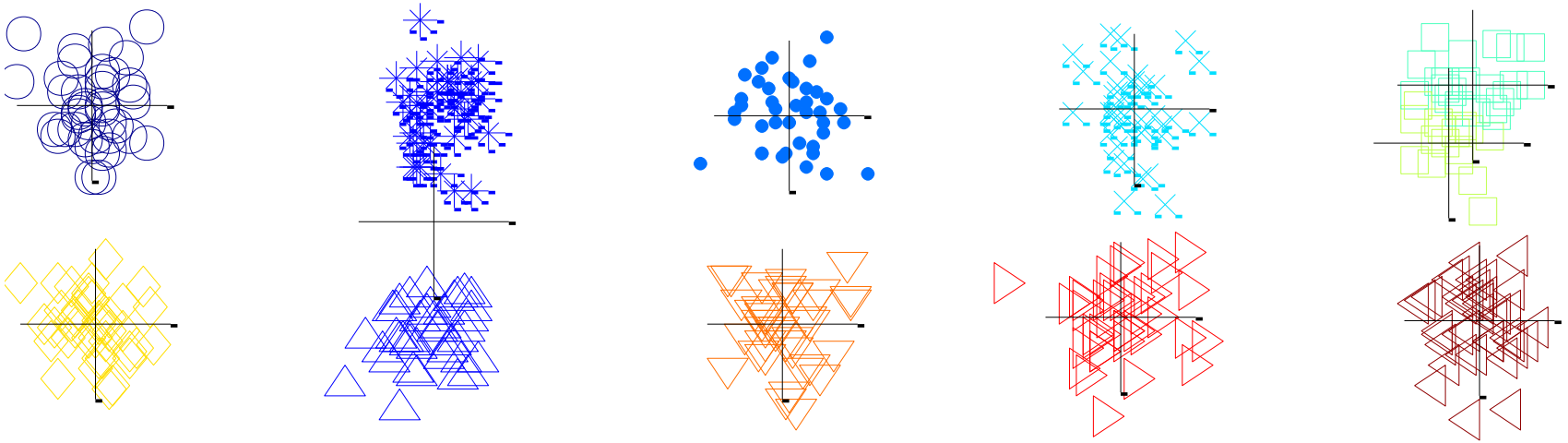
Starting with two initial centroids in one cluster of each pair of clusters

10 clusters example



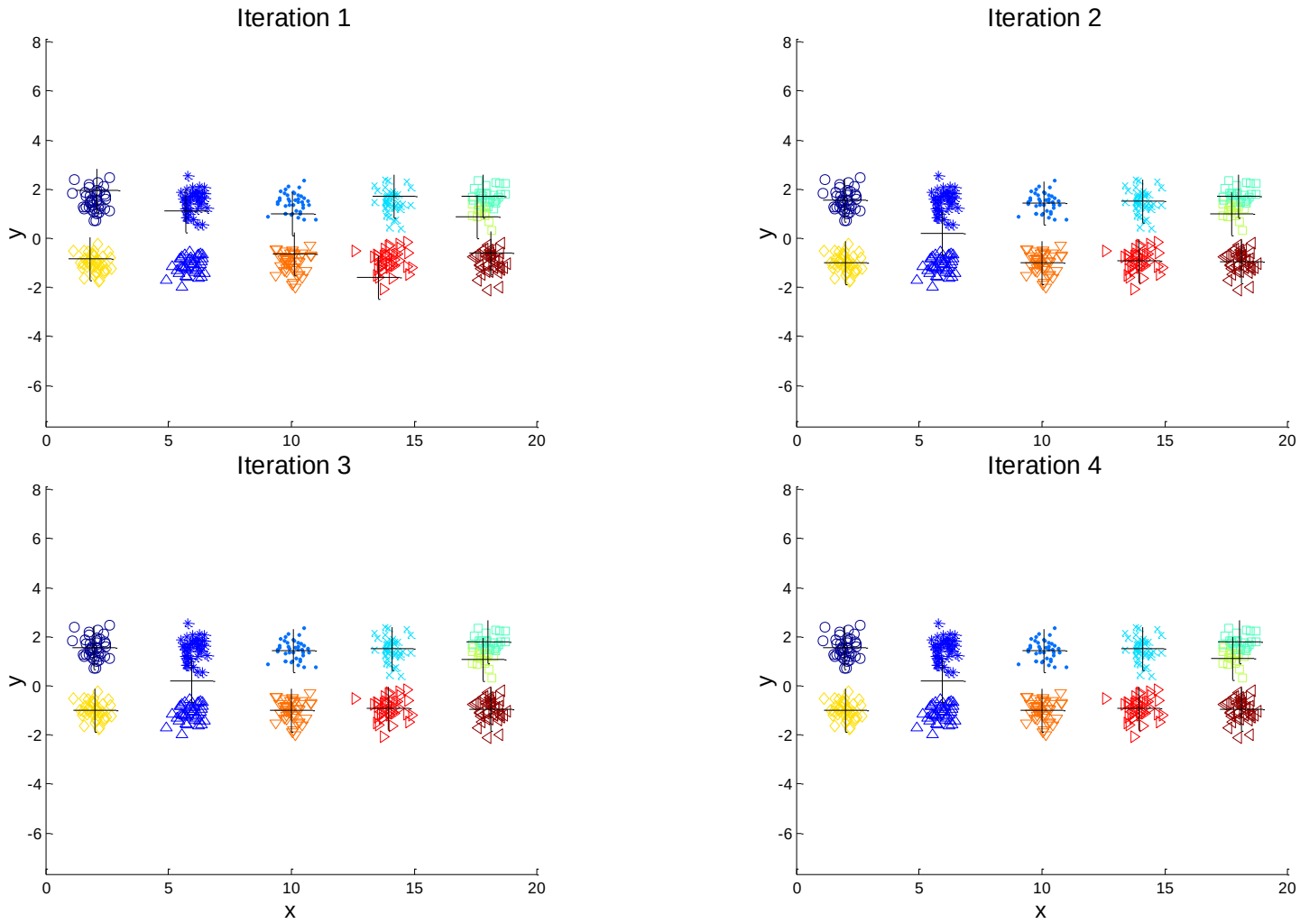
Starting with two initial centroids in one cluster of each pair of clusters

10 clusters example



Starting with some pairs of clusters having three initial centroids, while other have only one.

10 clusters example



Starting with some pairs of clusters having three initial centroids, while other have only one.

Dealing with initialization

- Do multiple runs and select the clustering with the smallest error

- Select original set of points by methods other than random
  - E.g., pick the most distant (from each other) points as cluster centers (K-means++ algorithm)

K-means++ algorithm

- Consistently produces better results in terms of SSE
  - By choosing the initial values (or "seeds") for the k-means clustering algorithm
- To make initialized centroids apart, perform the following

1. Select an initial point at random to be the first centroid
2. For k – 1 steps
   1. For each of the N points, x_i, $1 \le i \le N$, find the minimum squared distance to the existing centroids, $C_1, \ldots, C_j$, $1 \le j < k$, i.e., $\min_j d^2(C_j, x_i)$
   2. Randomly select a new centroid by choosing a point by probability $\frac{\min_j d^2(C_j, x_i)}{\sum_i \min_j d^2(C_j, x_i)}$
3. End For

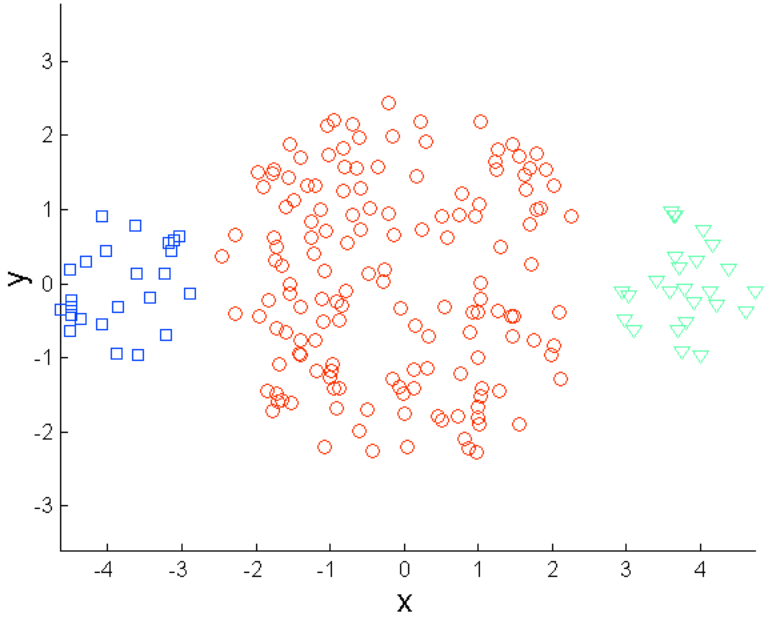
Make the new centroid far from the existing ones, and avoid choosing outliers!

Limitations of k-means

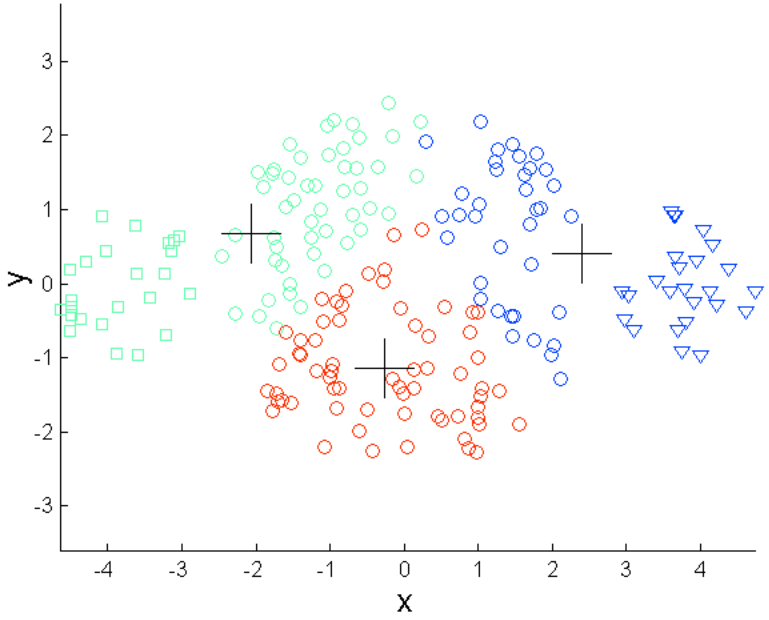
- K-means has problems when clusters are of differing
  - Sizes
  - Densities
  - Non-globular shapes

- K-means has problems when the data contains outliers.
  - One possible solution is to remove outliers before clustering

Limitations of k-means: differing sizes

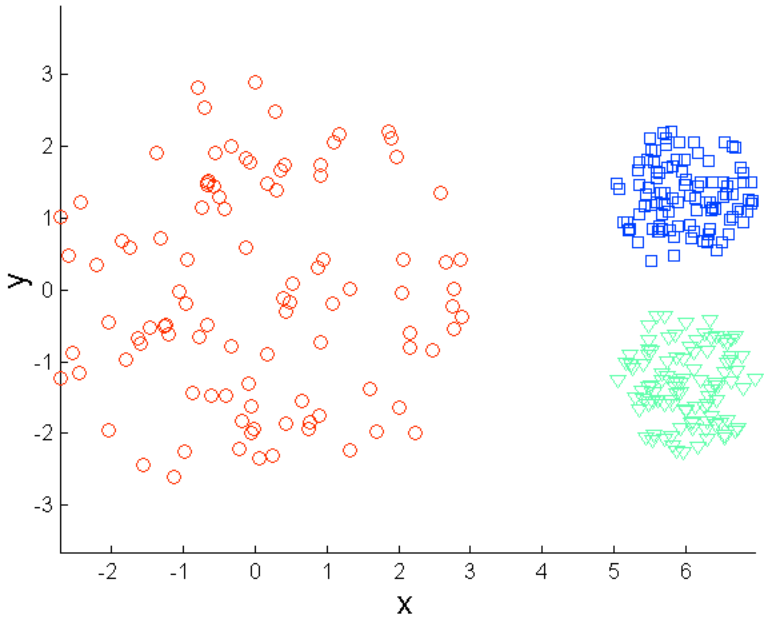


Original Points

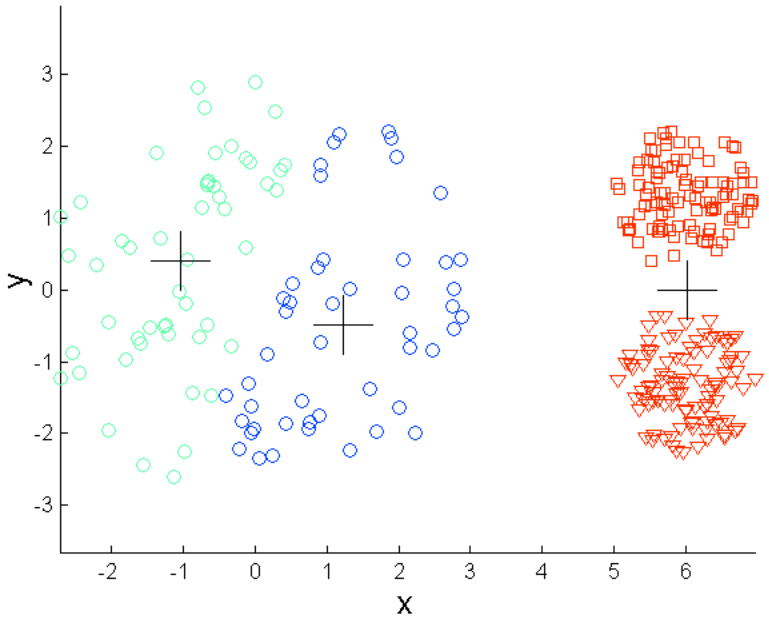


K-means (3 Clusters)

Limitations of k-means: differing density

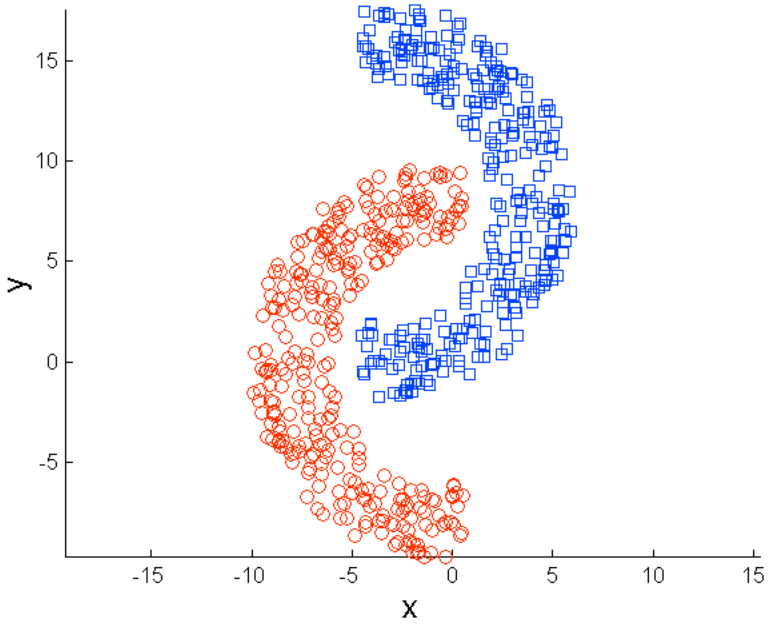


Original Points

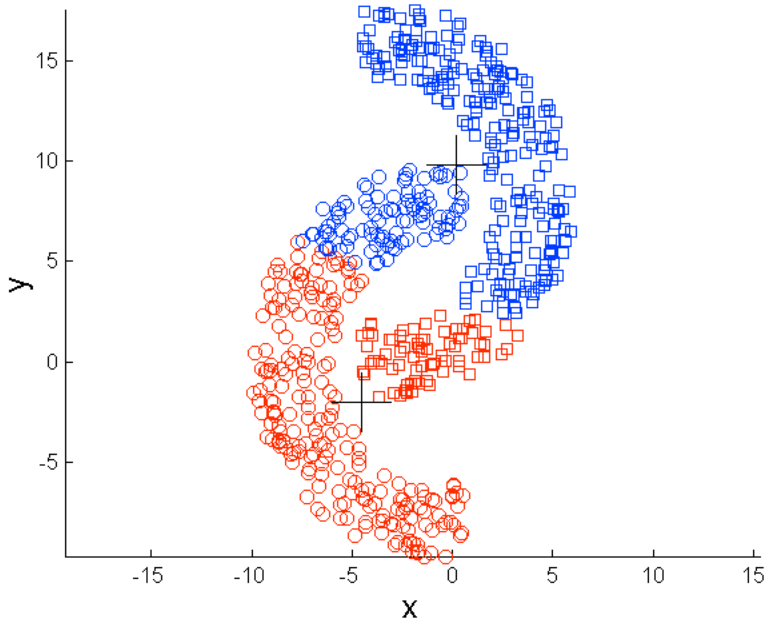


K-means (3 Clusters)

Limitations of k-means: non-globular shapes

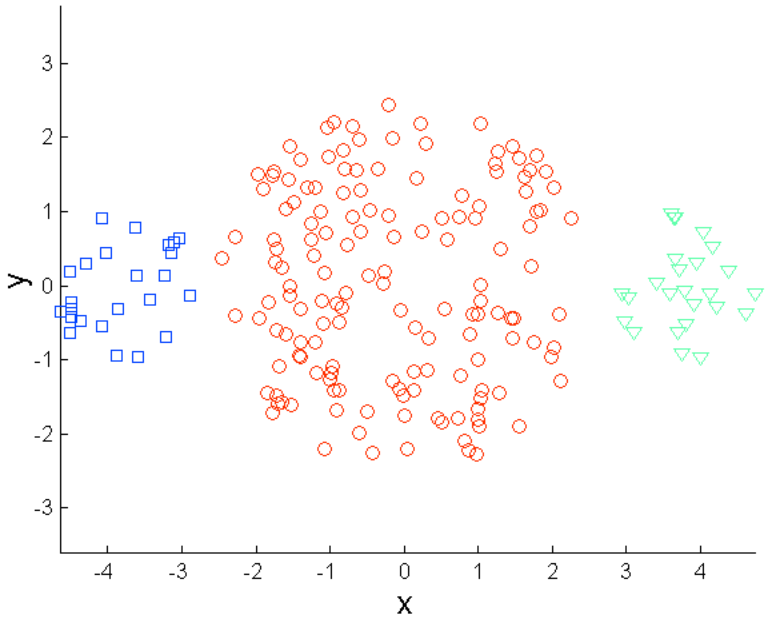


Original Points

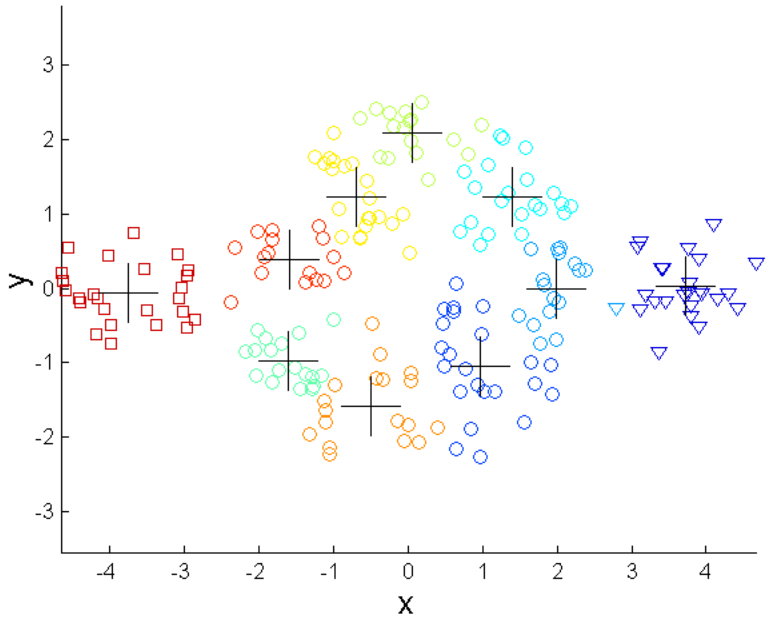


K-means (2 Clusters)

Overcome k-means limitations



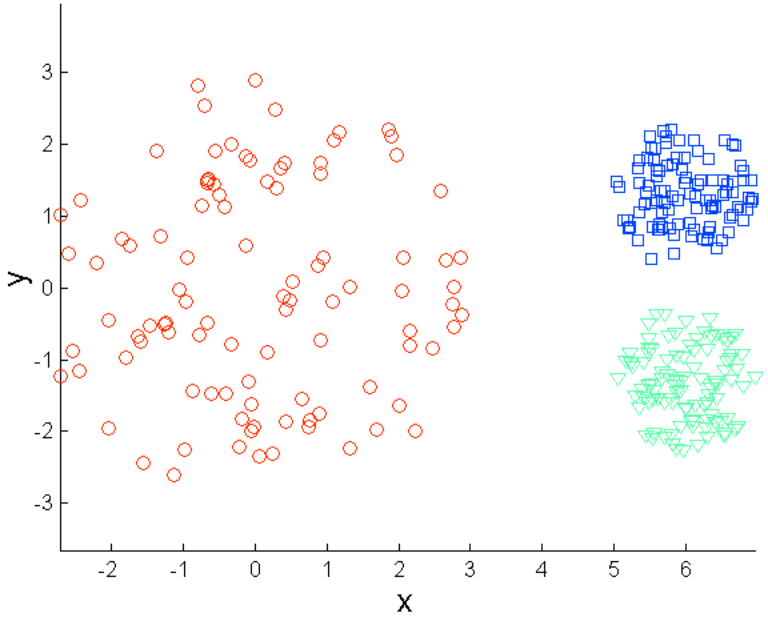
Original Points



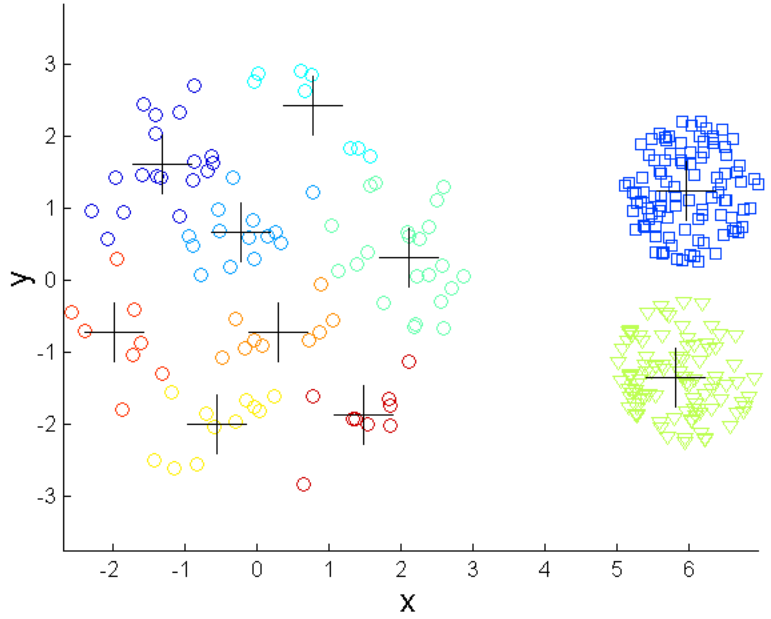
K-means Clusters

Find a large number of clusters such that each of them represents a part of a natural cluster. But these small clusters need to be put together in a post-processing step.

Overcome k-means limitations



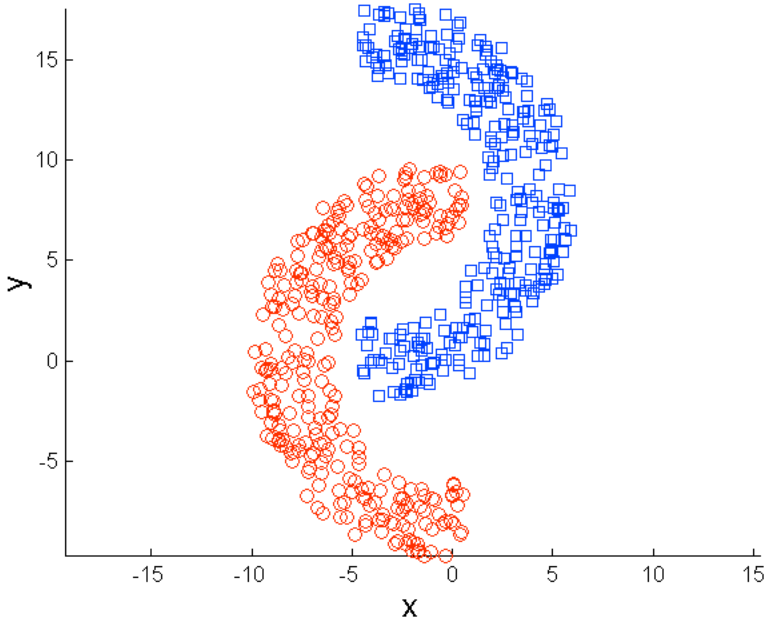
Original Points



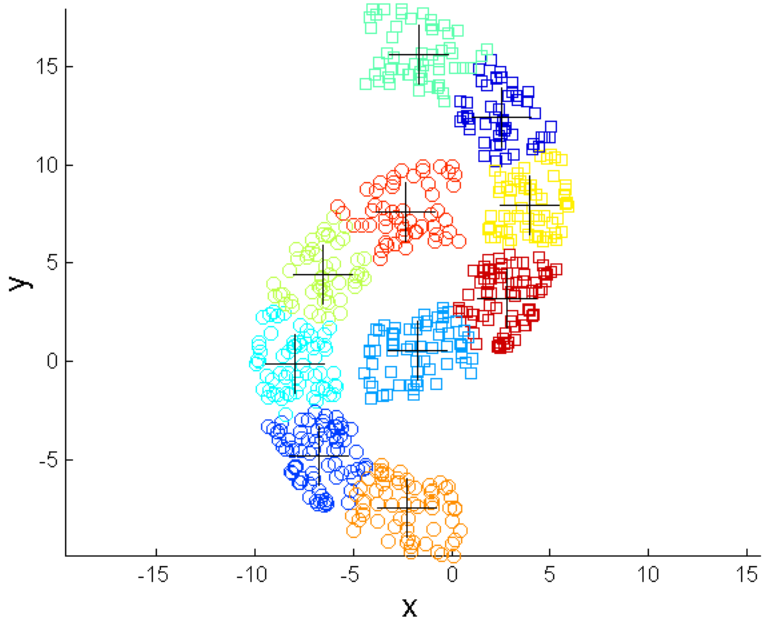
K-means Clusters

Find a large number of clusters such that each of them represents a part of a natural cluster. But these small clusters need to be put together in a post-processing step.

Overcome k-means limitations



Original Points



K-means Clusters

Find a large number of clusters such that each of them represents a part of a natural cluster.
But these small clusters need to be put together in a post-processing step.

Formulate k-means as a probabilistic model

- Recall bayes classifiers

$$p(\mathbf{x}, z) = p(\mathbf{x}|z)p(z)$$

- If z is not observed, we it a latent variable, or hidden variable
- By marginalizing out z, we get a density over the observable data $\mathbf{x}$

$$p(\mathbf{x}) = \sum_z p(\mathbf{x}, z) = \sum_z p(\mathbf{x}|z)p(z)$$

- It is latent variable model
- If $p(z)$ is a categorial distribution, this is a mixture model

Gaussian mixture models (GMM)

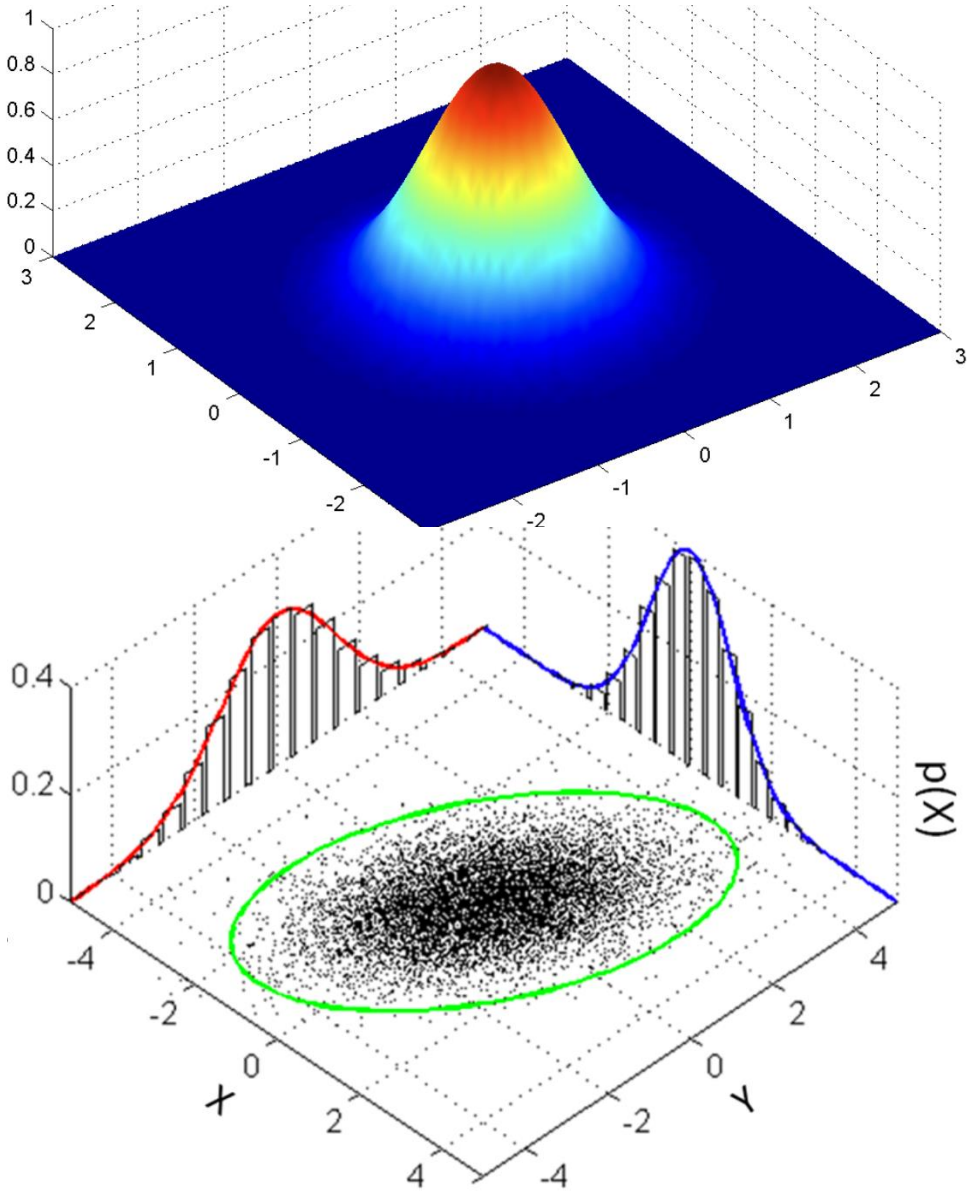
- A GMM represents a distribution as

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)$$

with π_k is the mixing coefficients (not the constant in Gaussian distribution)

$$\sum_{k=1}^{K} \pi_k = 1 \qquad \text{and} \qquad \pi_k >= 0 \quad \forall k$$

- It defines a density over x, so we can fit parameters using maximum likelihood.
  - We try to match the data density of x as closely as possible



Gaussian mixture models (GMM)

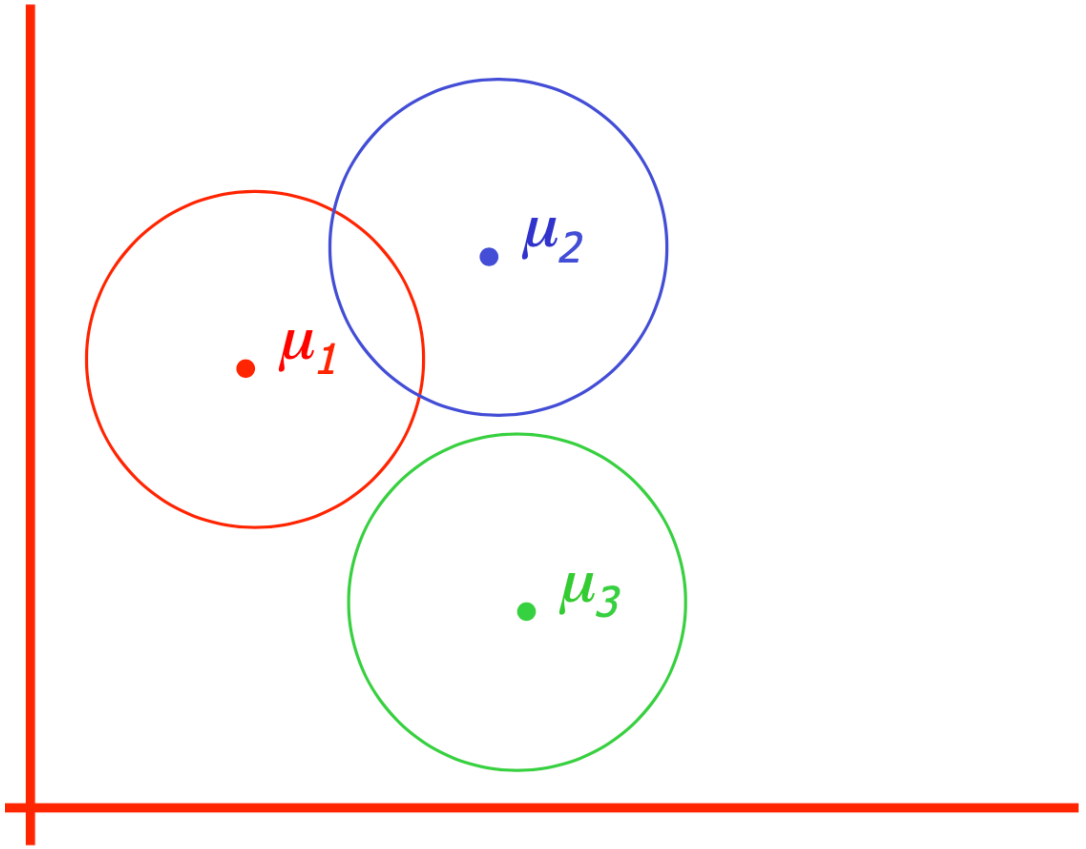
- We can also write the model as a generative process
- Data samples we observed can be generated through a series of probability distributions.
- For $i = 1, ..., N$:

$$z^{(i)} \sim \text{Categorical}(\pi)$$

$$\mathbf{x}^{(i)} \sim \mathcal{N}(\mu_{z^{(i)}}, \Sigma_{z^{(i)}})$$

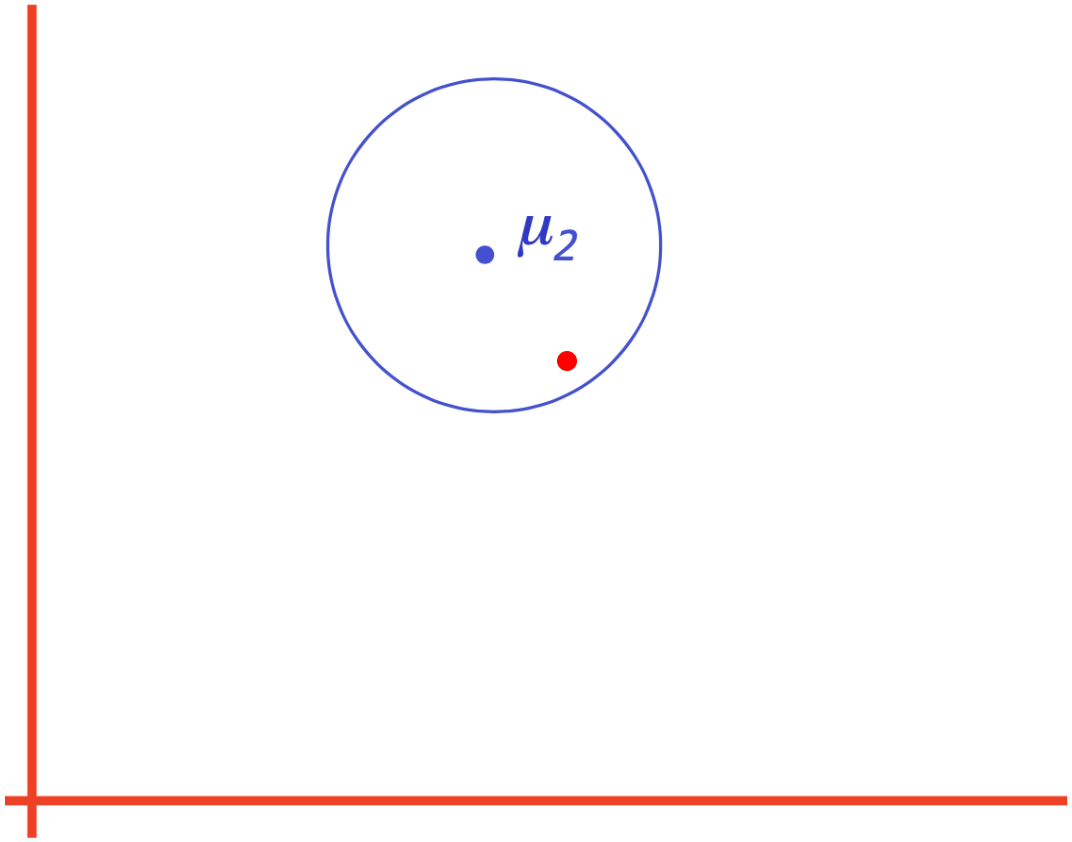
GMM generative process

- There are K components. The *k*-th component is called z_k
- Component z_k has an associated mean vector μ_k
- Each component generates data from a Gaussian with mean μ_k and covariance matrix Σ_k



Example

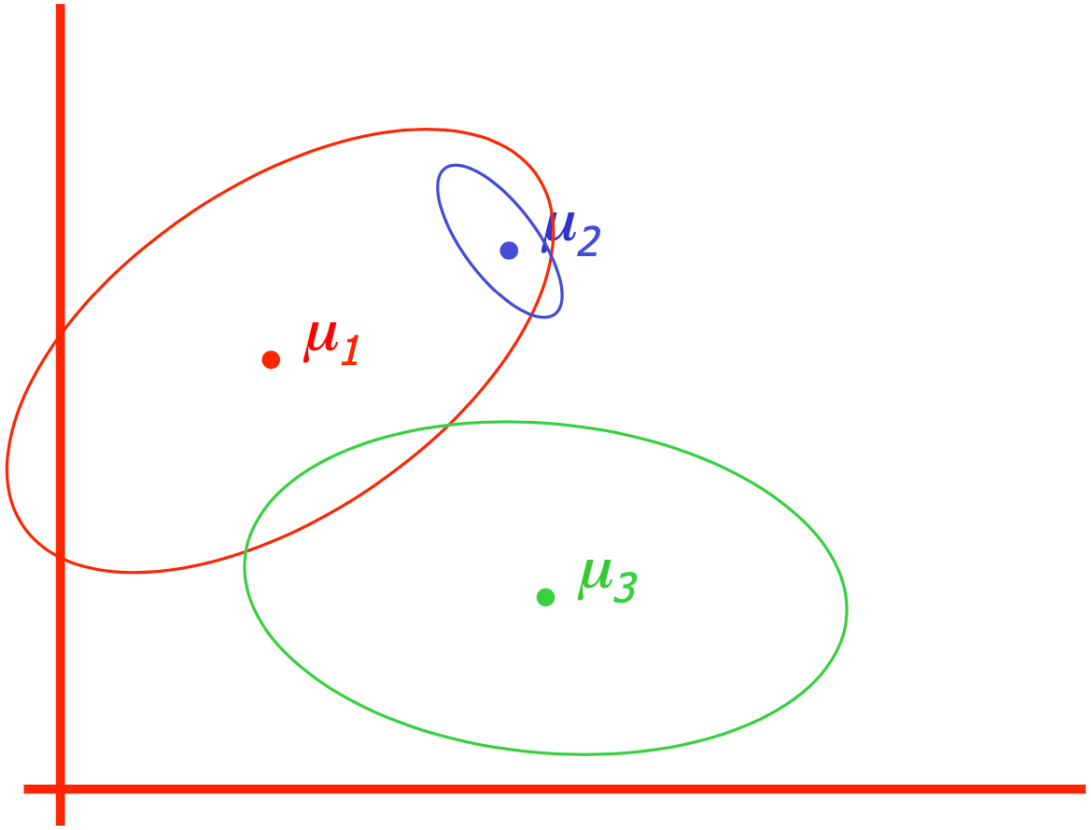
- Assume that each datapoint is generated according to the following recipe:
- 1. Pick a component at random. Choose component *k* with probability π_k.
- 2. Sample a datapoint ~ $\mathcal{N}(\mu_k, \Sigma_k)$



Example

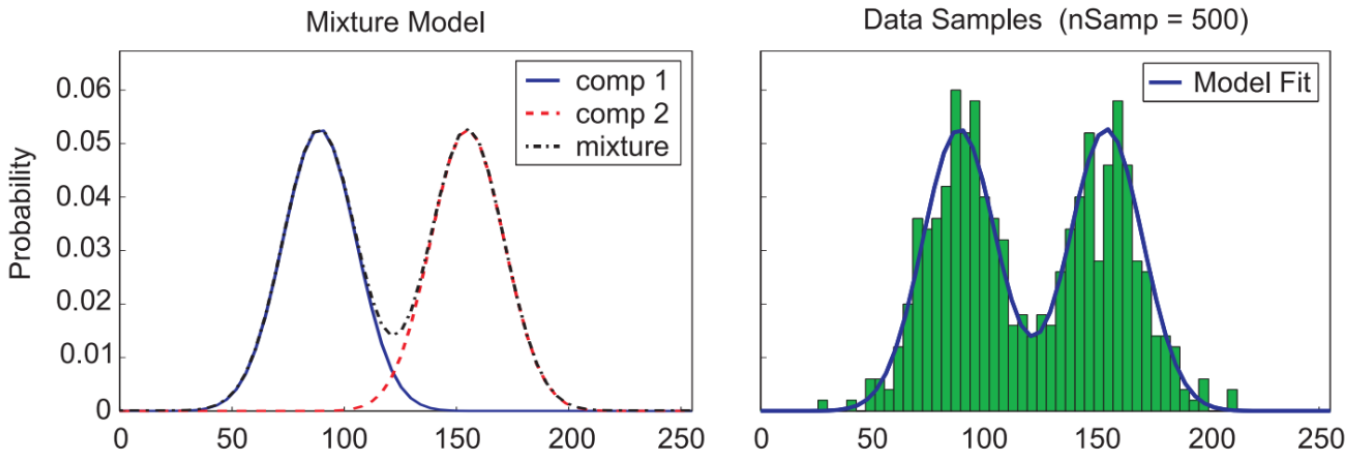
- Assume that each datapoint is generated according to the following recipe:
- 1. Pick a component at random. Choose component *k* with probability π_k.
- 2. Sample a datapoint ~ $\mathcal{N}(\mu_k, \Sigma_k)$

- Once we trained the parameters $\{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}$, the observed data can be generated by our distribution.

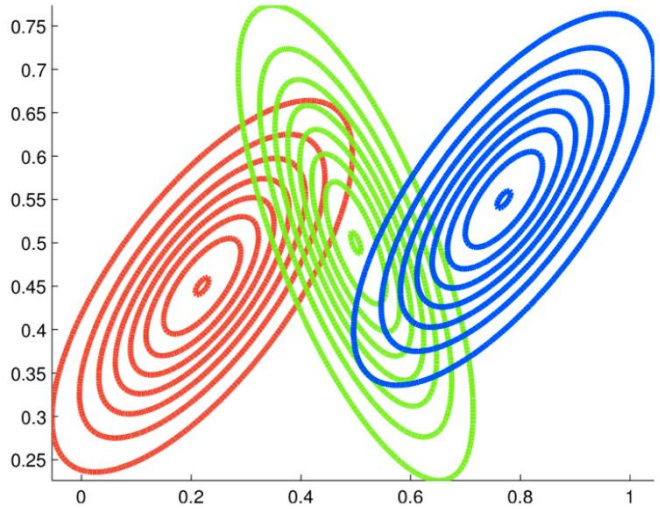


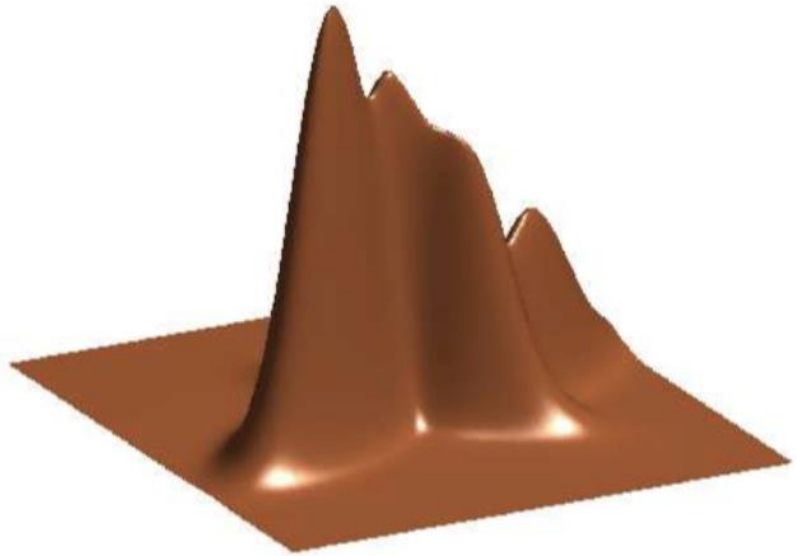
Visualizing a GMM

- 1D example (K=2)



- 2D example (K=3)





Learn GMM: maximum likelihood

- How to find the parameters $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}$
- Maximum likelihood principle: choose parameters to maximize likelihood of observed data
- We don't observe the cluster assignments z, we only see the data x
- Given data *D*, choose parameters to maximize

$$\log p(\mathcal{D}) = \sum_{n=1}^{N} \log p(\mathbf{x}^{(n)})$$

- We can find p(x) by marginalizing out z:

$$p(\mathbf{x}) = \sum_{k=1}^{K} p(z = k, \mathbf{x}) = \sum_{k=1}^{K} p(z = k) p(\mathbf{x} | z = k)$$

Optimize with latent variables

- But we don't know the datapoint assignment $z^{(i)}$, so we need to marginalize them out. Now the log-likelihood is difficult.

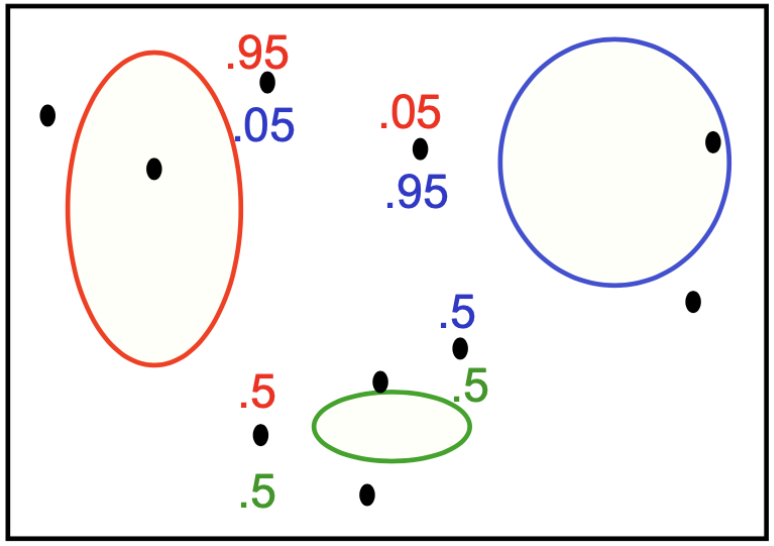
$$\log p(\mathbf{X}; \boldsymbol{\theta}) = \sum_{i=1}^{N} \log p(\mathbf{x}^{(i)} \mid \boldsymbol{\theta})$$

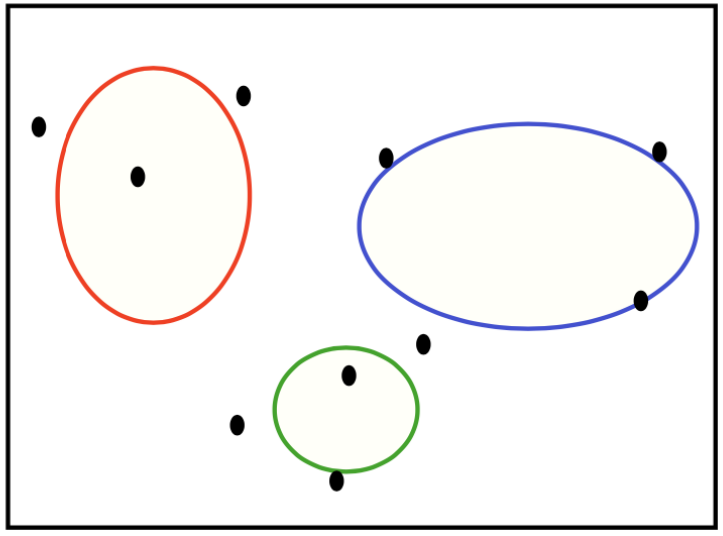
$$= \sum_{i=1}^{N} \log \sum_{z^{(i)}=1}^{K} p(\mathbf{x}^{(i)} \mid z^{(i)}; \{\boldsymbol{\mu}_k\}, \{\boldsymbol{\Sigma}_k\}) \, p(z^{(i)} \mid \boldsymbol{\pi})$$

- Problem: the log is outside the sum, so things don't simplify.
- We have a chicken-and-egg problem, just like with K-Means!
  - Given θ, inferring $z^{(i)}$ is easy.
  - Given $z^{(i)}$, learning θ (with maximum likelihood) is easy.
  - Doing both simultaneously is hard!

Expectation-Maximization (EM) algorithm

- EM algorithm alternates between two steps:
- Expectation step (E-step): Compute the posterior probability over z given our current data - i.e. how much do we think each Gaussian generates each datapoint.
- Maximization step (M-step): Assuming that the data really was generated this way, change the parameters of each Gaussian to maximize the probability that it would generate the data it is currently responsible for.





Example

- E-step:
  - Assign the responsibility $r_k^{(i)}$ of component *k* for data point *i* using posterior probability:

$$r_k^i = p(z^{(i)} = k|\mathbf{x};\theta)$$

- M-step:
  - Apply the maximum likelihood updates, where each component is fit with a weighted dataset. The weights are proportional to the responsibilities.

$$\begin{aligned}
\pi_k &= \frac{1}{N}\sum_{i=1}^{N} r_k^{(i)} \\
\boldsymbol{\mu}_k &= \frac{\sum_{i=1}^{N} r_k^{(i)} \cdot \mathbf{x}^{(i)}}{\sum_{i=1}^{N} r_k^{(i)}} \\
\boldsymbol{\Sigma}_k &= \frac{1}{\sum_{i=1}^{N} r_k^{(i)}}\sum_{i=1}^{N} r_k^{(i)}(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)^\top
\end{aligned}$$

K-means related to EM

- The K-Means Algorithm:
  1. Assignment step: Assign each data point to the closest cluster
  2. Refitting step: Move each cluster center to the average of the data assigned to it
- The EM Algorithm:
  1. E-step: Compute the posterior probability over z given our current model
  2. M-step: Maximize the probability that it would generate the data it is currently responsible for.